



RELATÓRIO TÉCNICO — ATIVIDADE AV1

Sistemas Operacionais - COMP0472 (2026.2 - AE)

Discentes: Augusto César Honorato dos Santos

Docente: Prof. Dr. Glauco de Figueiredo Carneiro

1. Introdução e Problema:

Aplicações de IA generativa executadas localmente mobilizam uma cadeia complexa de recursos do sistema operacional: processos, threads, memória, armazenamento, chamadas de sistema, sockets e escalonamento. Este trabalho tem como objetivo instalar e operar uma aplicação local baseada em Ollama, observar seu comportamento no Linux e medir o impacto de diferentes configurações de execução.

A pergunta norteadora é: **como a camada de aplicação, o modelo, a quantização e a configuração de execução afetam processos, threads, uso de CPU, memória e responsividade de um sistema local de IA generativa?**

2. Trilha Escolhida:

Trilha A — Chat local: Ollama + Open WebUI

O Ollama atua como runtime de inferência, expondo uma API HTTP local na porta 11434. O Open WebUI é a interface web, executada em container Docker, que se comunica com o Ollama via HTTP. Essa trilha é adequada porque permite observar claramente:

- Processos do Ollama e do container Docker;
- Comunicação HTTP pela API local;
- Portas, sockets e conexões;
- Volumes persistentes e logs;
- Comportamento sob múltiplas requisições simultâneas.

3. Modelo Selecionado

Item	Valor
Nome	llama3.2:1b
Família	Llama
Parâmetros	1.2 bilhões
Formato	GGUF
Quantização	Q4_0
Tamanho em disco	1.3 GB
Contexto máximo	131.072 tokens
Licença	Llama 3.2 Community License
URL Hugging Face	https://huggingface.co/meta-llama/Llama-3.2-1B

Justificativa: modelo leve, compatível com execução local em CPU, com bom equilíbrio entre qualidade e consumo de recursos. Permite realizar múltiplos experimentos sem estourar a memória da máquina virtual.

4. Ambiente Experimental

Item	Valor
Sistema Operacional	Ubuntu 24.04.2 LTS
Ambiente	Máquina Virtual (VirtualBox)
RAM total	3.916 MB (~3,8 GB)
Swap	0 MB
Disco /	91,75 GB (25,1% usado)
CPU	(preencher com lscpu)
Ollama	versão 0.34.0
Docker	versão 29.1.3
Open WebUI	container ghcr.io/open-webui/open-webui:main
IP da VM	10.0.2.15
IP do Docker	172.17.0.1

5. Arquitetura



O Ollama foi configurado para escutar em 0.0.0.0:11434 por meio de um arquivo de override do systemd:

```
/etc/systemd/system/ollama.service.d/override.conf
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

Isso permitiu que o container Docker alcançasse o Ollama através do host.

6. Processos, Threads e Chamadas de Sistema

6.1 Processos e threads

Durante a execução, foram identificados:

- ollama serve (PID 39747, PPID 1 — systemd): processo principal, escutando na porta 11434.
- llama-server (PID 42839): processo filho que efetivamente executa a inferência, com pico de **196% de CPU** e **37,4% de memória** (1,4 GB residentes).
- Container Docker** do Open WebUI, com processo Python interno escutando na porta 8080.

O strace -f mostrou que o Ollama opera com **10 threads simultâneas**, confirmando o caráter multithread da inferência.

6.2 Chamadas de sistema observadas

Durante uma requisição curta ("Oi"), o strace -c registrou 893 chamadas de sistema. As mais relevantes:

Syscall	% tempo	Chamadas	Papel
futex	49,38%	139	Sincronização entre threads
nanosleep	17,93%	257	Pausas curtas de threads
epoll_pwait	17,75%	61	Espera de eventos de I/O
write	2,16%	27	Escrita da resposta HTTP
read	1,71%	49	Leitura de dados e rede
openat	0,39%	17	Abertura de arquivos (pesos, logs)
connect / socket	0,72%	10	Comunicação de rede
madvise	1,53%	41	Gerenciamento de memória virtual

O predomínio de futex, nanosleep e epoll_pwait revela que o Ollama passa a maior parte do tempo **coordenando threads e aguardando eventos**, não executando cálculos puros.

7. Metodologia dos Experimentos

Foram realizadas **12 execuções mensuráveis**, divididas em três configurações:

- Configuração 1 — Entradas longas:** 4 perguntas técnicas abertas.
- Configuração 2 — Entradas curtas:** 4 perguntas triviais.
- Configuração 3 — Concorrência:** 4 pares de prompts executados simultaneamente.

Cada execução foi medida com o comando `time`, que reporta tempo real, tempo de usuário e tempo de sistema. O consumo de CPU e memória foi monitorado com `top`.

8. Resultados

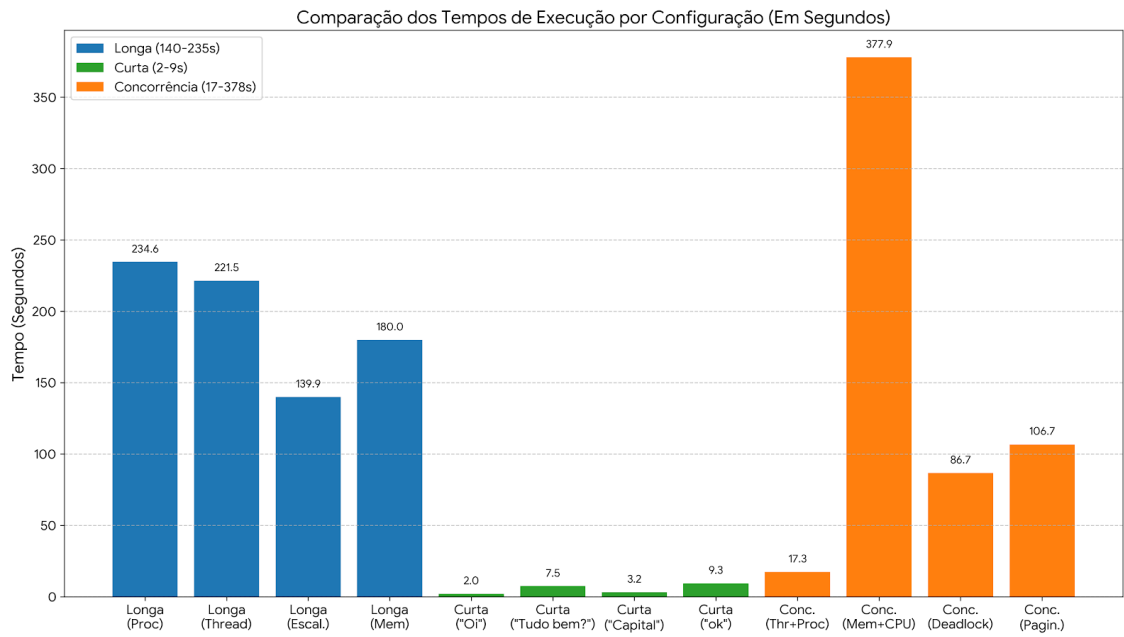
8.1 Tempos de execução

#	CONFIGURAÇÃO	PROMPT	TEMPO REAL
1	Longa	Explicar processo	3m54,6s
2	Longa	Explicar thread	3m41,5s
3	Longa	Escalonamento de CPU	2m19,9s
4	Longa	Memória virtual	3m00,0s
5	Curta	"Oi"	2,0s
6	Curta	"Tudo bem?"	7,5s
7	Curta	"Capital do Brasil"	3,2s
8	Curta	"Diga apenas: ok"	9,3s
9	Concorrência	Threads + Processos	17,3s
10	Concorrência	Memória + CPU	6m17,9s
11	Concorrência	Deadlock + Semáforo	1m26,7s
12	Concorrência	Paginação + Swapping	1m46,7s

8.2 Consumo de recursos

MÉTRICA	VALOR OBSERVADO
CPU DO LLAMA-SERVER (PICO)	196,2% (~2 núcleos)
MEMÓRIA DO LLAMA-SERVER (PICO)	37,4% (1,4 GB residentes)
RAM TOTAL	3.916 MB
RAM LIVRE DURANTE INFERÊNCIA	102–124 MB
LOAD AVERAGE (PICO)	1,81 / 1,15 / 2,23
SWAP UTILIZADO	0 MB

8.3 Gráfico principal



9. Discussão

Por que as entradas longas demoraram tanto? Porque o modelo precisa gerar muitos tokens, e cada token exige uma passada completa pela rede neural. Sem GPU, isso é feito em CPU, o que limita a vazão.

Por que a concorrência piorou o tempo? O Experimento 10 (6m17s) foi o mais lento de todos. Dois processos competindo por CPU e RAM ao mesmo tempo geram contenção: o escalonador do Linux alterna entre eles, mas ambos ficam mais lentos do que se rodassem sozinhos.

Por que a RAM é um gargalo? Com apenas 3,8 GB, o sistema ficou com ~100 MB livres durante a inferência. Sem swap, qualquer pico maior poderia causar falha. A escolha de um modelo de 1,3 GB foi decisiva para viabilizar os testes.

O modelo é confiável? Não. O llama3.2:1b apresentou confabulações graves:

- Disse que "thread" é uma conexão de rede.
- Disse que "memória virtual" é realidade virtual/simulação.
- Disse que "semáforo" é um dispositivo de trânsito.
- Disse que "swapping" é uma operação matemática.

Isso reforça a necessidade de **verificação crítica** de respostas geradas por IA, especialmente em modelos pequenos.

10. Relação com Sistemas Operacionais

Os experimentos ilustram, na prática:

- **Processos e threads:** o Ollama usa múltiplas threads sincronizadas por futex.
- **Escalonamento:** a concorrência evidenciou o custo do chaveamento de contexto.
- **Memória:** a ausência de swap e a RAM limitada tornaram o gerenciamento de memória crítico.
- **Chamadas de sistema:** read, write, openat, epoll_pwait e futex foram essenciais para carga de pesos, comunicação HTTP e sincronização.
- **Concorrência:** o aumento de requisições simultâneas não melhorou a vazão — piorou a latência.

11. Limitações

1. RAM muito baixa (3,8 GB) e ausência de GPU.
2. Ambiente virtualizado, com possível competição por recursos do host.
3. Modelo muito pequeno, com respostas frequentemente incorretas.
4. Alta variância nos tempos, dificultando comparações diretas.
5. Sem swap, o que limitou ainda mais a margem de manobra.

12. Declaração de Uso de IA Generativa

- **Ferramenta:** ChatGPT (OpenAI).
- **Finalidade:** apoio na interpretação de comandos Linux, estruturação do relatório e análise dos resultados.
- **Prompts relevantes:** (listar até 5)
- **Sugestões aproveitadas/corrigidas/rejeitadas:** (descrever)
- **Verificação:** todos os dados numéricos foram obtidos diretamente dos comandos time, top e strace, e conferidos nos logs brutos.

13. Conclusão

A atividade demonstrou que mesmo um modelo pequeno (1,2B parâmetros) exige recursos significativos quando executado em CPU sem GPU. O Ollama se mostrou eficiente em gerenciar threads e I/O, mas limitado pela RAM disponível. A concorrência não trouxe ganho de vazão e expôs o custo do escalonamento. Por fim, a qualidade das respostas do modelo reforça a importância de verificá-las criticamente, sobretudo em contextos técnicos.

14. Referências

- SILBERSCHATZ, A.; GALVIN, P.; GAGNE, G. *Operating System Concepts*. 10. ed. Wiley, 2018.
- TANENBAUM, A.; BOS, H. *Modern Operating Systems*. 5. ed. Pearson, 2023.
- KERRISK, M. *The Linux Programming Interface*. No Starch Press, 2010.
- OLLAMA. Repositório oficial. <https://github.com/ollama/ollama>
- OPEN WEBUI. Repositório oficial. <https://github.com/open-webui/open-webui>
- HUGGING FACE. Modelos. <https://huggingface.co/models>